METHODOLOGY REPORT · PSX DECOMPILATION

Decompiling PlayStation Games

A comparative study of four matching decompilation projects, synthesised into one workflow, a catalogue of hard rules, and working guidelines for byte-exact reconstruction of retro game source.

tenchu-decomp · lsddecomp · parasite-eve-decomp · rage-racer-decomp

COMPILED OCTOBER 2026

Table of Contents

1. Executive Summary 1

2. The Four Projects 2

2.1 tenchu-decomp — Tenchu: Stealth Assassins . . . 2

2.2 lsddecomp — LSD: Dream Emulator . . . 2

2.3 parasite-eve-decomp — Parasite Eve . . . 3

2.4 rage-racer-decomp — Rage Racer . . . 3

2.5 Side-by-side comparison . . . 4

3. The Workflow 5

3.1 Phase 0 — Groundwork and legal setup . . . 5

3.2 Phase 1 — Reconnaissance and toolchain identification . . . 6

3.3 Phase 2 — Bootstrap the byte-exact oracle . . . 6

3.4 Phase 3 — The matching loop . . . 7

3.5 Phase 4 — Quality beyond matching . . . 8

3.6 Phase 5 — Continuous verification and publishing . . . 9

4. Rules 9

4.1 Oracle integrity . . . 9

4.2 Toolchain rules . . . 9

4.3 Source and assembly rules . . . 10

4.4 Process and review rules . . . 11

5. Guidelines 11

5.1 Naming . . . 11

5.2 Code organisation and translation units . . . 12

5.3 Shiftability . . . 13

5.4 Documentation and knowledge management . . . 13

5.5 Guidelines for AI-assisted decompilation . . . 13

6. Starter Checklist for a New Project 14

7. References 15

1. Executive Summary

This report examines four open-source PlayStation 1 decompilation projects — *tenchu-decomp*, *lsddecomp*, *parasite-eve-decomp* and *rage-racer-decomp* — and distils their working methods into a single reference: a phase-by-phase workflow, a catalogue of hard rules, and a set of guidelines that cover naming, code organisation, tooling and the use of AI agents. All four projects pursue what the community calls a *matching decompilation*: C source that, when compiled with the original toolchain, reproduces the retail game executable byte for byte. That goal, more than any individual tool, is what shapes their methodology. A byte-exact target removes ambiguity from every debate, because a paraphrase of a program can be subtly wrong without anyone noticing, while a byte-exact rebuild cannot.

The survey surfaced a striking degree of convergence. Every project pins the original compiler version, splits the retail executable with the same family of tools, proves a disassembly-to-reassembly round trip before writing any C, and gates every change on a whole-image checksum. Every project is also *source-only*: no game data, no Sony SDK files and no compiler binaries are committed, so contributors must supply their own legally obtained copies. On top of this shared skeleton, each project layers its own discipline — Parasite Eve and Rage Racer formalise debt ratchets and evidence-based naming, LSD enforces its rules with git hooks, and Tenchu and LSD demonstrate that the entire process can be driven at scale by AI agents working against an automated oracle.

Chapters 2 through 5 present the analysis: chapter 2 profiles each repository, chapter 3 walks through the synthesised workflow, chapter 4 catalogues the rules the projects treat as non-negotiable, and chapter 5 collects the softer guidelines, including the orchestration patterns that made large-scale AI-assisted decompilation work. Chapter 6 condenses everything into a starter checklist for a new project. The headline numbers from the four repositories summarise what the methodology can achieve when followed consistently:

1,446
LSD functions matched, all of the game's own code (lsddecomp)

555 / 555
Tenchu game functions in matching C (tenchu-decomp)

1,200 / 1,200
Rage Racer functions, 423,420 code bytes (rage-racer-decomp)

106 rounds
LSD's AI-driven campaign, about 6,800 commits (lsddecomp)

> *A byte-exact oracle. An agent can't talk its way past a SHA1. Every claim was checked by the build rather than by the agent that made it.*
>
> lsddecomp — notes on AI-driven decompilation

2. The Four Projects

The four repositories differ in games, compilers and maturity, but they form a family: two of them share an author and a documentation system, two were driven primarily by AI agents, and all four build on the same open-source toolchain. This chapter profiles each project and closes with a comparison table.

2.1 tenchu-decomp — Tenchu: Stealth Assassins

Fuuzetsu's Tenchu project targets the PS1 stealth game *Tenchu 1 (Shinobi Gaisen)* and states its goal plainly: a byte-identical reimplementation in the spirit of the Super Mario 64 decompilation. At the time of writing, 537 of 555 game-code functions are fully decompiled C that byte-matches — the remaining 18 are canonical handwritten-assembly originals — covering 97.90% of game code bytes, and the split-then-reassemble round trip is verified byte-identical on every build. The build system is notable in its own right: a Haskell Shake driver (*./Build*) orchestrates an offline, reproducible nix environment, and the project maintains a second *shiftability* lane in which the game relinks through ordinary GNU ld with full relocation support, proving that code can grow across a 64 KiB boundary and still boot in the PCSX-Redux emulator.

Tenchu is also the most elaborate AI-assisted project of the four. A matching cookbook records the compiler idioms that reproduce the original bytes; an orchestration runbook describes how a main session spawns one *matcher* agent per function in isolated git worktrees, harvests only green results, and folds every newly learned rule back into the cookbook. A dozen small tools (triage, findsimilar, siblingdiff, matchdiff, autorules, permute and others) measure the tree rather than trusting prose, and the docs explicitly instruct agents to record durable knowledge in the shared docs directory instead of private memory. Name recovery leans on the demo disc's PSX.SYM file, which preserved original source file names, prototypes and struct layouts.

2.2 lsddecomp — LSD: Dream Emulator

brian-oblivion's lsddecomp is the completed proof of the methodology: on 29 September 2026, 32 days after the first split, every one of the game's 1,446 functions was matching C and the rebuilt executable was byte-for-byte identical to retail *SLPS_015.56*. The campaign was run as an explicit experiment in AI-driven decompilation: Claude agents wrote every function, header, name and comment, while the human operator supplied the disc and SDK, set the goal and made the ownership calls. Each round had a head agent planning and measuring, up to five runner agents each working one source file in its own git worktree, cheaper models on mechanical work, and the strongest model revising the plan. The whole process record — round logs, per-function match reports, research notes — lives on an *archive/process* branch.

The repository is as famous for its rules as for its result. Its CLAUDE.md opens with six hard rules enforced by git hooks: never edit the retail hash files, always build through *build-and-verify.sh*, never commit anything of Sony's, never edit generated disassembly, treat the pinned GCC 2.6.3 toolchain as part of the retail bytes, and never fix register allocation with inline-asm pins outside the sanctioned GTE macro layer. Its documentation distils the campaign's lessons, including a celebrated section on *how a score lies* — stale builds, leftover assembly stubs and address drift can all make a diff look greener than it is. Sony's Psy-Q libraries are not decompiled at all: they are linked from Sony's own objects, with the SDK disc provenance of every object measured against retail and recorded in configuration.

2.3 parasite-eve-decomp — Parasite Eve

khasinski's Parasite Eve project is a work-in-progress matching decompilation of the North American release (*SLUS_006.62*) and is the most systematically documented of the four. Its docs directory defines a full contract: CONVENTIONS governs naming and style, CODE_ORGANIZATION governs translation units and shiftability, and an extensive ASM_AND_GTE_POLICY regulates when assembly may appear in source. The game is a harder target than LSD — it loads 162 room overlays and 22 scene sets into a shared RAM window, each linked separately against the main executable's symbols — and the build treats overlays as first-class citizens with their own manifests and checks.

Two ideas stand out. First, progress is measured as *verified semantic C*, not merely matching bytes: a unit earns credit only when its complete binary matches retail SHA-1, and register pins, empty barriers and text-resident data are tracked separately as crutch debt. Second, ratchet discipline: CI fails if debt, placeholder-name or organization counts increase, and an improvement may not leave a stale higher baseline behind. Every cleanup is therefore permanent. The review contract requires each change to state which claim it makes — binary match, semantic recovery, typing, naming or organization — because a green byte check proves only the first of those.

2.4 rage-racer-decomp — Rage Racer

khasinski's Rage Racer project demonstrates the methodology at completion scale and across regions. The PAL executable matches all 1,200 reported functions and all 423,420 code bytes, and the same tree produces byte-exact USA and Japanese builds by reusing the PAL C wherever the compiler output translates directly — 328 of 360 USA units reuse PAL source unchanged, with genuine regional differences isolated in their own files. The project is candid about source forms: its report partitions units into plain C, C with inline assembly or compiler constraints, and units retaining assembly or raw opcodes, so a reader can see exactly how much of the tree is reconstructed C versus disclosed assembly wrapped in HANDWRITTEN_ASM macros.

Rage Racer also contributes the cleanest treatment of naming evidence. Its docs/names.md records what each function and global is and why, source files are named after their subject rather than whichever function sits first in them, and handwritten-assembly origin is inferred from instruction idioms and

calling conventions — with the report explicitly warning that a macro's name is not itself evidence of the original source language. The regional port tooling rejects any gap or overlap in the executable's text range before a build is even attempted, and every regional target is verified against its own retail SHA-1.

2.5 Side-by-side comparison

Project	Game and target	Compiler toolchain	Status	Signature discipline
tenchu-decomp	Tenchu 1 (Shinobi Gaisen), main executable plus six variants	GCC 2.8.1 cc1 (decompals), maspsx, GNU ld, nix-pinned, Shake driver	Game code 555/555 matched; 97.90% bytes	AI matcher agents + cookbook; dual exact/shiftability lanes
lsddecomp	LSD: Dream Emulator, SLPS-01556, single PS-X EXE	Psy-Q GCC 2.6.3, maspsx, binutils, splat, pinned	Complete: 1,446/1,446 functions, retail byte-identical	Git-hook-enforced hard rules; build-and-verify oracle
parasite-eve-decomp	Parasite Eve (USA disc 1), SLUS-006.62 plus 162 room overlays	GCC 2.7.2 cc1 (old-gcc), maspsx, Psy-Q 4.0, binutils	In progress; main EXE and overlays under matching	Debt ratchets; verified semantic C; evidence order
rage-racer-decomp	Rage Racer PAL SCES-006.50 plus USA and two Japan builds	Old PSX GCC variants, maspsx, binutils	Executable matching complete; 1,200/1,200 functions	Source-form partition reporting; naming evidence file

Table 1. The four surveyed projects and their distinguishing practices.

Reading the table column by column reveals the family resemblance. The compiler column always names an exact old GCC revision with maspsx reproducing Sony's assembler behaviour — never a modern compiler, and never an unpinned one. The status column separates matching (bytes) from finishing (names, types, organisation), a distinction every project draws explicitly. And the discipline column shows four different answers to the same question: how do you stop contributors, human or artificial, from making the build green by cheating?

3. The Workflow

Despite four different codebases and four different maintainers, the projects follow one recognisable lifecycle. Figure 1 shows it as six phases: the numbering is chronological, but only phase 4 is a true loop — everything else is traversed once per project (and re-run per binary or per region where a game has several targets). The subsections after the figure describe each phase and the practices that support it.



Figure 1. The synthesised PSX decompilation workflow, from legal groundwork to continuous verification.

3.1 Phase 0 — Groundwork and legal setup

Every project begins by deciding what the repository will and will not contain, and the answer is identical in all four: *source only*. No disc images, no extracted assets, no FMV, no Sony Psy-Q SDK files, no compiler binaries and no generated split output are ever committed. Contributors supply their own legally obtained copies, and build scripts read them from git-ignored directories (*disk/*, *sdk/*, *assets/*). The rebuild is a clean-room reconstruction: LSD dedicates its own original work to the public domain under CC0 while noting that the game and the SDK remain Sony's; Parasite Eve and Rage Racer state the same split in their licence files. This posture keeps the projects publishable on GitHub and is a precondition for everything else, because CI must be able to clone and audit the tree without ever touching copyrighted material. LSD additionally reads its private dependencies (the retail executable and converted SDK libraries) from a separate private repository at CI time, so the public pipeline still runs end to end.

3.2 Phase 1 — Reconnaissance and toolchain identification

The second phase turns a retail disc into a fully characterised build target. Each project records the target executable's name and SHA-1 in its README as the definition of success — Parasite Eve pins *SLUS_006.62* with entry PC 0x80072534 and text VRAM 0x80010000 to 0x801FE000; Rage Racer tabulates four regional executables with their hashes, entry points and payload sizes. Reconnaissance also maps the memory layout: whether the game is a single plain executable (LSD) or a resident executable plus overlays loaded into fixed RAM windows (Parasite Eve), because that decision drives the entire build architecture.

Toolchain identification is the quietly decisive step. PSX compilers left fingerprints, and the projects identify the exact compiler revision — Tenchu builds with GCC 2.8.1's cc1, Parasite Eve with GCC 2.7.2, LSD with the Psy-Q GCC 2.6.3 — plus the Psy-Q library versions the game links. maspsx reproduces Sony's ASPSX assembler macro expansion, and binutils supplies the linker. Where luck provides extra evidence, it is mined systematically: Tenchu extracts the demo disc's PSX.SYM to recover original file names, prototypes and struct layouts, and audits its own declarations against those records before trusting them. The phase ends with a written statement of facts (flags, load address, $gp value, SDK object list) that later phases treat as fixed.

3.3 Phase 2 — Bootstrap the byte-exact oracle

Before anyone writes a line of C, the project proves it can rebuild the game from its own disassembly. The splat tool splits the executable into per-function assembly, data, a linker script and symbol lists; the pinned toolchain then reassembles everything; and the build asserts that the result is byte-identical to retail. Tenchu's docs call this round trip the foundation of the whole project, and LSD's setup script refuses to finish unless the first build already matches. Only against this green base does C work begin: functions are converted from assembly stubs to C one at a time, so any regression is attributable to exactly one change.

Phase 2 also installs the verification gate that the rest of the project lives inside. LSD compresses it into a single script — build, then compare the whole image with retail, ending with *OK: build matches retail* — and a hook blocks any bare *make* that would bypass it. Parasite Eve exposes the same idea as *make verify* (source policy, tests, build, retail checksum) and *make verify-clean* for structural changes that must not trust stale objects. The core toolchain components shared across the projects are summarised in Table 2.

Component	Role in the pipeline	Used by
splat	Splits the retail executable into per-function assembly, data, symbols and linker scripts	All four projects
maspsx	Translates Psy-Q/GCC assembly output for GNU as, reproducing Sony's ASPSX expansions	All four projects
old GCC cc1 (2.6.3 / 2.7.2 / 2.8.1)	The pinned original compiler that makes byte matching possible	Matched to each game
m2c / Ghidra	First-draft C generation from assembly for a function under match	Tenchu, LSD, community default
asm-differ / matchdiff / funcdiff	Per-function byte diffs against retail during iteration	All four projects
decomp-permuter	Automated source-shaping search for residual register-allocation ties	All four projects, as a last resort
objdiff / decomp.dev	Whole-tree progress reports, badges and per-function status	All four projects

Table 2. Core toolchain components and their roles.

3.4 Phase 3 — The matching loop

Matching is the heart of the project and the only phase that repeats thousands of times. A round begins with target selection, and the projects are unanimous that selection is a tool job, not a taste job: Tenchu's triage script rates every unmatched function by difficulty and names the nearest already-matched twins; the guidance is to pick small, clean-seam functions first and to map a new subsystem's shared structs once so that whole families become cheap. The draft is seeded mechanically (Ghidra's decompiler plus m2c's output in Tenchu's pipeline), then shaped by hand against the compiler's known idioms.

Iteration follows a strict ladder, summarised in Table 3, because experience showed that fixing diffs in the wrong order wastes enormous effort. Length mismatches mean the structure is wrong; operand mismatches are meaningless until length agrees; register differences usually fall out of earlier fixes. Only after deterministic work does the decomp-permuter get one bounded run, and some residuals are declared below the obvious C level and parked honestly as NON_MATCHING rather than chased with surgical sessions. A close draft that resists matching is preserved behind *#ifdef NON_MATCHING* with an INCLUDE_ASM fallback, so the image stays byte-identical while the honest state of the function is visible in the source.

Order	Fix first	Why
1	Length: instruction count must match exactly	A different count means the structure (dispatch, loops, temps) is wrong; everything else is noise until length agrees
2	Wrong, missing or reordered instructions	Map each difference to a known compiler idiom family; read the nearest matched siblings first
3	Register-only differences	Usually fall out of steps 1-2; steer with statement order and temps before anything heavier
4	Mechanical rules via tools	Run the rule sweeper on any compiling draft; if no known rule wins, the residual is something new
5	One bounded permuter run	Only for right-length, small residuals; never as a decompiler, and never for GTE macro functions
6	Park it honestly	Sub-C residuals are recorded NON_MATCHING with a status header instead of burning days

Table 3. The matching iteration ladder, synthesised from the Tenchu cookbook and project rules.

The loop closes with the only acceptance test that matters: the whole-image checksum. Tenchu commits only on a green *./Build check*; LSD's head agent re-verified every claimed match against the full-image SHA1 before merging anything; Parasite Eve requires *make verify* before any hand-off. A per-function diff may say MATCH, but only the linked image promotes the result — the projects learned this the hard way through stale builds and leftover assembly stubs that scored full matches against themselves.

3.5 Phase 4 — Quality beyond matching

All four projects insist that matching is one quality level, not the finish line. Parasite Eve's CONVENTIONS defines a file as *finished* only when it is byte-matched *and* fully named, typed, free of hidden fixed addresses, and placed in its subsystem; a file that meets only the first condition is *matched, not finished*. The work at this phase is deliberate cleanup: replace func_ and D_ placeholders with evidence-backed names, convert raw offsets into typed struct fields, define data tables as C, and drive the readability debt counters (magic numbers, raw offsets, unknown fields) toward zero. LSD ran this as a distinct final campaign with its own plan and tooling; Rage Racer records the evidence for every name in a dedicated document so the reasoning survives the people who made it.

Organisation is treated with the same evidentiary care as the bytes themselves. Parasite Eve's evidence order for placing or merging code runs from addresses, section layout and manifest order, through known object and library boundaries, to shared data and call relationships, and only last to filenames, which are hints rather than evidence. Translation-unit merges must keep addresses and manifest order unchanged, remain contiguous, and survive a clean byte-identical rebuild. Nothing about the tree's shape is allowed to be an aesthetic choice.

3.6 Phase 5 — Continuous verification and publishing

The final phase keeps the project honest over years. CI on every push rebuilds the image, checks it against the retail hash, and publishes an objdiff report that decomp.dev renders as per-function progress; Parasite Eve's CI additionally audits the report and refuses to publish when committed badges disagree with it. Lint jobs enforce formatting, documentation coverage and the project's declaration rules. Ratchets make regressions impossible to hide: debt counts, placeholder-name counts and organization counts may go down but never up, and raising a baseline is a policy change that needs explicit review. Badges in the README are regenerated from the audited report rather than typed in, because numbers written by hand go stale within a round.

4. Rules

Each project separates its hard rules — violations block a change mechanically — from its guidelines. This chapter catalogues the hard rules with their rationale, because every one of them was paid for: each exists because someone, or some agent, broke it and the build or the tree suffered. They fall into four groups: oracle integrity, toolchain, source and assembly, and process.

4.1 Oracle integrity

The retail hash is the project's constitution. LSD states it most bluntly: never edit *check.sha1* or *build.sha1*, because changing one to make a build pass deletes the oracle — and a git hook enforces it. A bare *make* says nothing about whether the bytes are right, so builds run only through the verify script, and another hook blocks the bare command. Structural changes (shared headers, the compiler wrapper, the Makefile, manifests, translation-unit boundaries) must pass a clean rebuild, since header dependencies protect incremental builds but do not replace a clean acceptance check. The generalisation, worth quoting as the field's golden rule:

> **The oracle is not negotiable.** Every claimed result is checked by the build, not by the author of the claim. Hash files are immutable, verify scripts are the only build entrance, and a green per-function diff never substitutes for the whole-image checksum.

4.2 Toolchain rules

The compiler is part of the retail bytes, not a knob to turn. The toolchain is pinned to exact revisions — compiler, binutils, flags and maspsx — and final builds must use the unmodified stock tools: no local patches, no diagnostic compiler builds, no post-build rewrite passes, no build hacks. Parasite Eve draws the boundary sharply: if stock C with allowed constraints does not match, the function remains an

assembly subsegment until its source reconstruction is corrected, and extending the assembler wrapper is not an acceptable matching technique. LSD adds the process rule for suspected toolchain faults: report them to the operator with a minimal reproducer, never experiment on the pinned tools. Per-file compiler options are permitted only as stock flags declared in file markers, and Tenchu explicitly forbids per-source variants of the build contract — fixed layout and allocator policy have one authority so the linker tools can consume it.

4.3 Source and assembly rules

The source rules exist because the original compiler is re-running on modern hardware, and it is intolerant of style. Table 4 collects the rules that appear across the projects with their rationale. Two deserve emphasis. First, register pinning and empty asm barriers are treated as debt, not as tools: allowed when they reproduce the original allocator's result while control flow stays in C, counted by the debt ratchet, and removed after matching whenever the match survives without them. Second, whole-function assembly is never a decompilation result — a generated INCLUDE_ASM stub remains the correct representation of an unmatched function, and inline instructions appear only through evidenced macros for operations with no C spelling (the GTE/COP2 layer) or for routines proven to have been assembly originally, with every such use listed in an evidence file.

Rule	Rationale	Where it is enforced
Never edit generated disassembly (asm/)	splat regenerates it; manual edits are silently overwritten and corrupt the oracle	LSD hook; all projects by convention
Never edit the retail hash files	Changing the target to pass a build deletes the oracle	LSD git hook
Functions stay in ROM-address order within a file	GCC emits sections in source order; an out-of-order function shifts later bytes	Code review; LSD and Tenchu docs
C89 discipline: declarations at block top, /* */ comments only	The original compilers require it; later syntax changes code generation	All projects
A rodata string is defined once, by name	A literal written at a use emits a second copy and shifts the image	LSD docs; Parasite Eve conventions
No register pins or extended-asm operands as matching shortcuts	They encode no recovered logic and hide real understanding; tracked as crutch debt	Parasite Eve debt ratchet; LSD rule 6; Tenchu anti-rule
Inline assembly only via evidenced macros (GTE/COP2, proven-asm functions)	COP2 has no C spelling; everything else has one and must use it	gte.h allowlists; ASM_AND_GTE_POLICY; evidence JSON files
No program addresses in C; refer by symbol	Fixed addresses break shiftability and hide layout knowledge	Parasite Eve shift audit; Tenchu ram_layout authority

Rule	Rationale	Where it is enforced
Compiler and maspsx are stock and pinned	Byte matching is only possible against the exact original toolchain	CI builds; setup scripts
Renames go through the rename tools, never by hand	Tools update every reference and re-verify; hand edits miss sites	LSD tools/rename.py; Tenchu config flow

Table 4. Hard source and assembly rules, with the reasoning that produced them.

4.4 Process and review rules

The process rules govern how changes move through the tree. Commit only on green: Tenchu's orchestrator cherry-picks agent work only after a whole-image clean rebuild passes, and harvesting fixes authorship metadata deliberately. Keep experiments out of the tracked tree: a C file enters *src/* only together with the manifest entry that configures its range, so a fresh clone can never diverge from CI. State your claim: Parasite Eve requires every change to declare whether it asserts a binary match, semantic recovery, typing, naming or organization, because the byte check validates only the first. Ratchets must equal the current tree: a debt baseline that no longer matches reality is a bug, and an improvement that leaves a stale higher baseline behind is rejected so cleanups cannot be silently spent later. Finally, knowledge lives in the repository, not in private memory: Tenchu records durable lessons in its docs directory so humans and agents on other machines benefit, and LSD keeps its CLAUDE.md free of any number that could go stale — measure with tools instead of transcribing counts.

5. Guidelines

Guidelines are the conventions a project applies by review and ratchet rather than by hook. They are where the projects encode their taste — and, in every case, the taste is for evidence, longevity and readability by the next contributor, human or artificial.

5.1 Naming

Parasite Eve states the principle that all four projects follow: *only PsyQ names are original*. Sony SDK functions keep their exact SDK spellings; every other name is a project name chosen by responsibility and may be improved as understanding improves. Names describe what code does for the game, not how it does it and not where it was found; types, sizes and identified addresses are never encoded into names. Placeholder names such as func_ and D_ are migration state rather than a style, and their counts are ratcheted so they only go down. Rage Racer adds the evidence dimension: a dedicated document records what each function and global is and why, so naming claims are auditable. Table 5 reproduces the working naming scheme.

Kind	Form	Example
Game function	Prefix_VerbObject, PascalCase after a subsystem prefix	Scene_LoadRoom, Inv_InsertItem
PsyQ function	Exact SDK spelling	SpuSetReverb, CdControl
Unknown function	func_XXXXXXXX (address, upper-case hex)	func_80192F9C
Global variable	g_PascalCase	g_GameState, g_PlayTimeFrameCounter
Unknown global	D_XXXXXXXX	D_800966EC
File-local data / function	s_PascalCase data; Prefix_Name functions	sTitleTimPath
Type (struct, typedef)	PascalCase, no _t suffix	Pe1GameState, FieldActor, AkaoTrack
Enum constant / #define	UPPER_SNAKE with a subsystem prefix	INV_CMD_GET_REMAINING_AMMO
Header / C file	Named after the responsibility of the unit, not its first function	menu_widget.h, Battle_EncounterInitializationFlow.c

Table 5. The naming scheme, after Parasite Eve's CONVENTIONS and Rage Racer's names.md.

5.2 Code organisation and translation units

The organising insight, stated in Parasite Eve's CODE_ORGANIZATION, is that the repository is a long-lived reconstruction of the original program, not a collection of independent decompilation puzzles. Directories map to subsystems with one responsibility and one function prefix each; overlays live in their own directory per binary; and a shared library linked into many overlays is stored once and listed by each overlay's manifest. Headers follow a strict ownership rule: one owner per declaration, in the narrowest appropriate header, with no per-function headers and no new file-scope extern declarations in C files. Moving a declaration must preserve its exact width, signedness and prototype — centralising is not an invitation to fix an uncertain ABI.

Translation-unit boundaries are treated as claims about the original source, and the evidence order governs them: executable addresses, section layout, relocations and manifest order come first; known object or archive boundaries next; then shared data, call relationships and static state; then responsibility; and only last existing filenames, which are hints. A merge or split is accepted only when addresses and manifest order survive, the unit is contiguous (unless object evidence proves otherwise), ownership is clear, and a clean rebuild stays byte-identical. Large cohesive units are acceptable — function count and aesthetics alone never justify inventing boundaries.

5.3 Shiftability

Tenchu and Parasite Eve elevate a subtle property to a first-class goal: shiftability, the ability of code and data to change size without breaking anything else. A matching build proves the bytes; shiftability proves the source does not depend on where those bytes sit. The practical rules are simple to state: refer to code and data by symbol; never write a program address in C (no casts of integers to pointers, no tables of addresses typed as integers); fixed addresses are allowed only for hardware registers, the scratchpad and the loader contract, each behind a named macro; and data the program owns is defined in a translation unit and placed by the linker. Both projects back the rules with measurement — Parasite Eve's shift test relinks the image with padding inserted and classifies every word that did not move, producing a category-by-category inventory of remaining absolute pins and a planned removal order. The payoff is modding, porting and research builds that relink rather than patch.

5.4 Documentation and knowledge management

Documentation in these repositories is engineered against the same failure mode as everything else: staleness. LSD's rule file is kept deliberately short because it holds rules and the loop, never project state or counts — a number written in prose is right for one session and wrong for every one after; state lives in tools that measure the tree. Durable lessons go into a cookbook with their evidence function attached, and the cookbook warns that every rule is a hypothesis that has been paid for at least once; when a rule and a measurement disagree, the measurement wins and the rule gets fixed. Comments in source explain why: what the code is for in the game, which evidence fixes a boundary, why an odd construct exists for the match — never narration, history or scores, which belong in commit messages. Process history is archived rather than deleted: LSD keeps round logs and per-function reports on an archive branch, and Tenchu moves superseded investigation logs out of source files verbatim so the code reads like source.

5.5 Guidelines for AI-assisted decompilation

Two of the four projects were driven end to end by AI agents, and their retrospectives converge on a set of guidelines that generalise well beyond decompilation. The foundation is the oracle: an agent cannot talk its way past a SHA1, so every claim is verified by the build rather than by the agent that made it, and hooks block the obvious escape routes (editing hash files, editing generated code, bypassing the verify script). The second foundation is measurement: project state lives in tools, never in prose, because transcribed numbers go stale within a round and send later agents to redo finished work — a wrong score is corrected the next time anyone measures, but a wrong diagnosis is what the next round acts on, and wrong diagnoses stood for many rounds before a screen caught them.

The orchestration patterns are equally concrete. Work is bundled one function (or one tight family) per agent in an isolated git worktree, with prompts generated by tooling rather than handwritten so that facts, twins and evolving guidance stay current. The orchestrator reviews, harvests on green, and folds newly

learned rules back into the shared cookbook; a reflection pass builds a tool whenever friction recurs. Models are tiered — cheaper models take mechanical work, stronger models take matching and naming, and the strongest revises the plan between rounds — and economics are tracked well enough that Tenchu's runbook can state which function classes cost two to three times more for half the yield, and should simply be deferred. Finally, the rules grew with the agents: LSD's working documents grew to thousands of lines as each round wrote down what the last one got wrong, then were distilled repeatedly. The lesson the project draws is that discipline does not constrain the agents — it is what makes them fast.

6. Starter Checklist for a New Project

For a reader inspired to begin a new PSX decompilation, the four repositories suggest the following sequence. It compresses the workflow of chapters 3-5 into ten concrete steps; the tool names in parentheses are the community defaults the projects share.

1. Pick one retail revision and record its executable name, SHA-1 and entry point. Every address in every config is wrong for any other revision.

2. Create a source-only repository. Gitignore the disc dump, SDK files and all generated output; document what contributors must supply.

3. Identify the original compiler and Psy-Q library versions, and pin them exactly (old GCC cc1 plus maspsx and binutils; prebuilt compilers exist in the decompme/compilers and decompals/old-gcc collections).

4. Configure splat and split the executable; commit the config and symbols, never the generated output.

5. Prove the disassemble-to-reassemble round trip is byte-identical before writing any C. Do not proceed until this gate is green.

6. Install the verification gate: one script that builds and compares the whole image against retail, plus hooks that block bypasses.

7. Adopt the loop: pick easy targets with matched twins first, map a subsystem's structs before its functions, and iterate length-first per Table 3, parking honest NON_MATCHING stubs.

8. Write the rules file early — oracle, toolchain, source, process — and keep it short; move measurements into tools and lessons into commit messages.

9. Set up CI that rebuilds, verifies the SHA-1 and publishes an objdiff report; add debt ratchets the day the first placeholder name is introduced.

10. Plan the cleanup phase from the start: naming evidence, typed structures, subsystem organisation and shiftability are what turn a matched binary into a finished, readable reconstruction.

7. References

The primary sources for this report are the four repositories themselves, cited with the documents consulted; tooling is listed where the projects credit it. All repositories were accessed in October 2026.

Fuuzetsu. *tenchu-decomp: Decompilation of the Tenchu 1 (Shinobi Gaisen) PlayStation 1 game*. https://github.com/Fuuzetsu/tenchu-decomp (README; CLAUDE.md; docs/README.md; docs/matching-cookbook.md; docs/orchestration.md; PLAN.md).

brian-oblivion. *lsddecomp: A matching decompilation of LSD: Dream Emulator*. https://github.com/brian-oblivion/lsddecomp (README.md; CLAUDE.md; docs/ARCHITECTURE.md).

khasinski. *parasite-eve-decomp: Matching decompilation of Parasite Eve (USA)*. https://github.com/khasinski/parasite-eve-decomp (README.md; CONTRIBUTING.md; docs/CONVENTIONS.md; docs/CODE_ORGANIZATION.md; docs/ASM_AND_GTE_POLICY.md).

khasinski. *rage-racer-decomp: Byte-exact source reconstruction of Rage Racer (PAL, USA, Japan)*. https://github.com/khasinski/rage-racer-decomp (README.md).

ethteck. *splat: The community splitting tool*. https://github.com/ethteck/splat.

mkst. *maspsx: MIPS assembler translation layer reproducing Sony ASPSX behaviour*. https://github.com/mkst/maspsx.

matt-kempster. *m2c: MIPS assembly to C decompiler*. https://github.com/matt-kempster/m2c.

simonlindholm. *asm-differ* and *decomp-permuter*. https://github.com/simonlindholm/asm-differ; https://github.com/simonlindholm/decomp-permuter.

encounter. *objdiff*, and the decomp.dev reporting platform. https://github.com/encounter/objdiff; https://decomp.dev.

decompme. *compilers: prebuilt historical compiler collection*. https://github.com/decompme/compilers; decompals. *old-gcc*. https://github.com/decompals/old-gcc.